

21CSC301T

Formal Language and Automata

UNIT I

FINITE AUTOMATA AND REGULAR EXPRESSION

Agenda of this Session

- Introduction to Automaton

Introduction to Automaton

- The mathematical theory of a simple class of algorithms that are important in computer sciences and mathematics is called the theory of automata.
- It has primarily two applications in computer science
 1. Modeling of Finite State Systems
 2. Description of regular sets of finite words
- It has many emerging applications such as
 - Computer algorithms and programs
 - Verification and specification of protocols
 - Designing of compilers
 - Searching for patterns in texts
 - Natural Language Processing

Introduction to Automaton (Cont..)

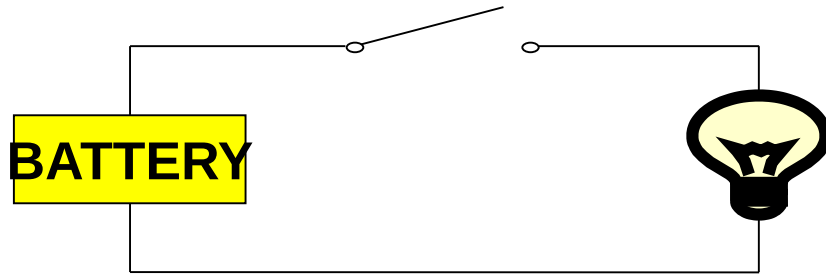
- Algorithms are recipes that tell us how to solve problems
- Initially, mathematicians were interested in the dividing line between what was and what was not algorithmic
- There are two approaches to classify algorithms
 1. According to their running times and is known as **complexity theory**
 2. According to the types of memory used in implementing them and is known as **language theory**

Automata – What is it?

- The term “Automata” is derived from the Greek Word "αὐτόματα" which means “self-acting”
- An automaton (Automata in plural) is an abstract self-propelled computing device which follows a predetermined sequence of operations automatically
- Automata theory deals with the logic of computation with respect to simple machines, referred to as automata
- Through automata, computer scientists are able to understand how machines compute functions and solve problems and more

Automata – What is it? (Cont..)

- Automata are abstract models of machines that perform computations on an input by moving through **a series of states or configurations**
- At each state of the computation, a transition function determines **the next configuration** on the basis of a finite portion of the present configuration
- Once the computation reaches an **accepting configuration**, it accepts that input
- Example : Electric Switch



Input	: switch
Output	: light bulb
Actions	: flip switch
States	: on, off

- Other examples : Fan Regulator, Automatic door controller

Families of Automata

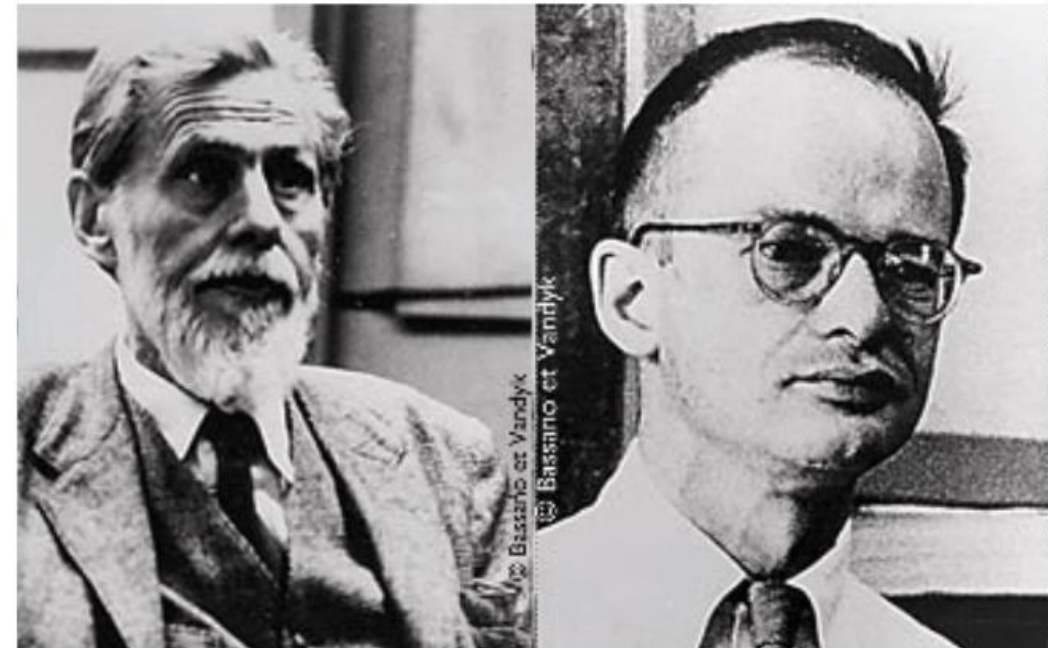
- There are four major families of automaton:
 - Finite Automata – Regular Languages
 - Pushdown Automata – Context free
 - Linear-bounded Automata – Context sensitive
 - Turing Machine – Recursively enumerable
- Finite Automata is the simplest automata and Turing Machine is the most complex and the most powerful automata

History of Automata Theory

- This was originated in 1930s, before the existence of modern computers
- Well-renowned computer scientist Alan Turing conceived the first “infinite” (or unbounded) model of computation: the Turing machine in 1936
- Warren McCulloch and Walter Pitts, two neurophysiologists, were the first to present a description of finite automata in 1943
- Later, two computer scientists, G.H.Mealy and E.F.Moore generalized the theory to much more powerful machines



Alan Turing ([source](#))



Warren McCulloch and Walter Pitts ([source](#))

Agenda of this Session

- Formal Languages: Strings, Languages, Properties

Alphabet

- An alphabet is a finite set of symbols denoted by Σ
- Examples:

$$\Sigma = \{a, b\}$$

$$\Sigma = \{0,1\} \text{ is a binary alphabet}$$

$$\Sigma = \{a, b, c \dots z, A, B, C \dots Z\} \text{ is a character alphabet}$$

$$\Sigma = \{0,1,2, \dots 9\}$$

String

- A **string** over some alphabet is a finite sequence of symbols drawn from that alphabet.

Example: $\Sigma = \{0,1\}$ Strings = $\{011, 100, 0101, 111, \dots\}$

- The term **sentence** and **word** are often used as synonyms for the term “string”.
- The **length of a string** s , written as $|s|$ is the no. of occurrences of symbols in s .
- E.g. banana is a special string of length 6.
- The **empty string** ε is a special string of length 0.
- The reverse of a string u is denoted as u^R

Example: $u = \text{abab}$ $u^R = \text{baba}$

Some common terms associated with parts of a String

Terms for Parts of Strings

The following string-related terms are commonly used:

1. A *prefix* of string s is any string obtained by removing zero or more symbols from the end of s . For example, **ban**, **banana**, and ϵ are prefixes of **banana**.
2. A *suffix* of string s is any string obtained by removing zero or more symbols from the beginning of s . For example, **nana**, **banana**, and ϵ are suffixes of **banana**.
3. A *substring* of s is obtained by deleting any prefix and any suffix from s . For instance, **banana**, **nan**, and ϵ are substrings of **banana**.
4. The *proper* prefixes, suffixes, and substrings of a string s are those, prefixes, suffixes, and substrings, respectively, of s that are not ϵ or not equal to s itself.
5. A *subsequence* of s is any string formed by deleting zero or more not necessarily consecutive positions of s . For example, **baan** is a subsequence of **banana**.

Kleene Closure

- The set of all strings including the empty string on the alphabet is written as Σ^*
- This is also known as Kleene closure or Kleene star after the famous logician who was one of the founders of this concept
- For example

$$\Sigma = \{ 1 \}, \text{ then } \Sigma^* = \{ \epsilon, 1, 11, 111, \dots \}$$

$$\Sigma = \{ a, b \}, \text{ then } \Sigma^* = \{ \epsilon, a, b, ab, ba, aaa, bbb, abab, baba, bbbbaa, \dots \}$$

Language

- The term **language** denotes any set of strings over some fixed alphabet.
- Let Σ be an alphabet and Σ^* be the set of all strings on the alphabet Σ . Any subset of Σ^* is called a language over Σ or language with alphabet Σ
- Generally, language is denoted by L and $L \subseteq \Sigma^*$
- **Abstract languages** like $\varnothing$, the empty set or $\{\epsilon\}$, the set containing only empty string are language under this definition.

Examples:

- All the syntactically well formed pascal programs form a language.
- All grammatically correct English sentences form a language.

Operations on Language

- There are several important operations that can be applied to languages.

OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

- The exponentiation operator to languages can be generalized by defining L^0 to be $\{\varepsilon\}$ and L^i to be $L^{i-1}L$.
- Thus L^i is L concatenated with itself i-1 times.

Concatenation of a string

- If x and y are strings, then the concatenate of x and y , written xy is the string formed by appending y to x .
- For example:

$x = \text{dog}$

$y = \text{house}$

$xy = \text{doghouse}$

- The empty string is the identity element under concatenation.

i.e., $s \varepsilon = \varepsilon s = s$

String Exponentiation

- Define s^0 to be ε and for $i > 0$ define s^i to be $s^{i-1} s$.
- Since εs is s itself, $s^1 = s$

Then

$$s^2 = ss$$

$$s^3 = sss \text{ and so on}$$

Example

Let L be the set $\{A, B, \dots, Z, a, b, \dots, z\}$ and D the set $\{0, 1, \dots, 9\}$

1. $L \cup D$ is the set of letters and digits.
2. LD is the set of strings consisting of a letter followed by a digit.
3. L^4 is the set of all four letter strings.
4. L^* is the set of all strings of letters including ϵ , the empty string.
5. $L(L \cup D)^*$ is the set of all strings of letters and digits beginning with a letter.
6. D^+ is the set of all strings of one or more digits.

Agenda of this Session

- Finite Automata
- Deterministic Finite Automata

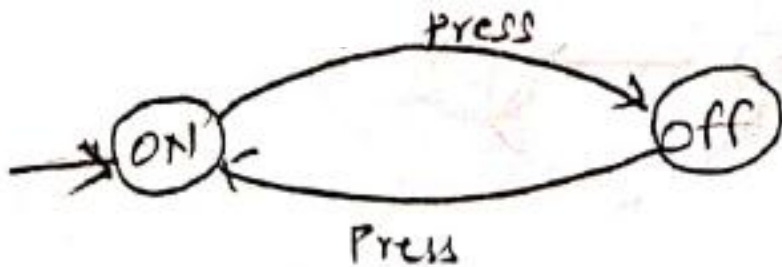
Finite State Automata

- Finite automata is a machine that given an input, jumps through a series of states according to a transition function
- It is a model of behaviour composed of states, transitions and actions
- Finite state machines are widely used in
 - Modelling of application behaviour
 - Design of hardware digital systems
 - Software engineering
 - Study of computation and languages

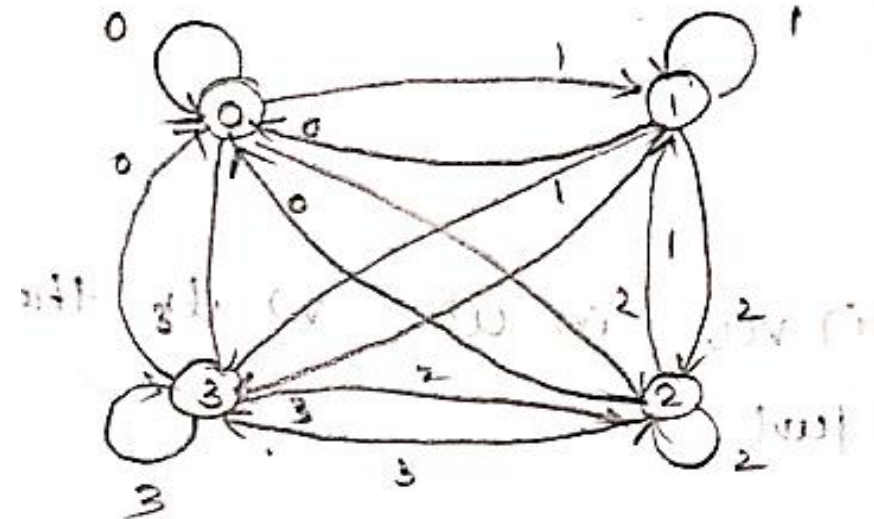
Finite State Automata (Cont..)

- Finite automata is a mathematical model of a system with discrete inputs and outputs
- It consists of a finite set of states and a set of transitions that occur on input symbols chosen from an alphabet set Σ
- One of these states is called the initial state from where the automata begins. There can be one or more final states.
- **Examples:**

Operation of a switch



Operation of an elevator



Types of Finite Automata

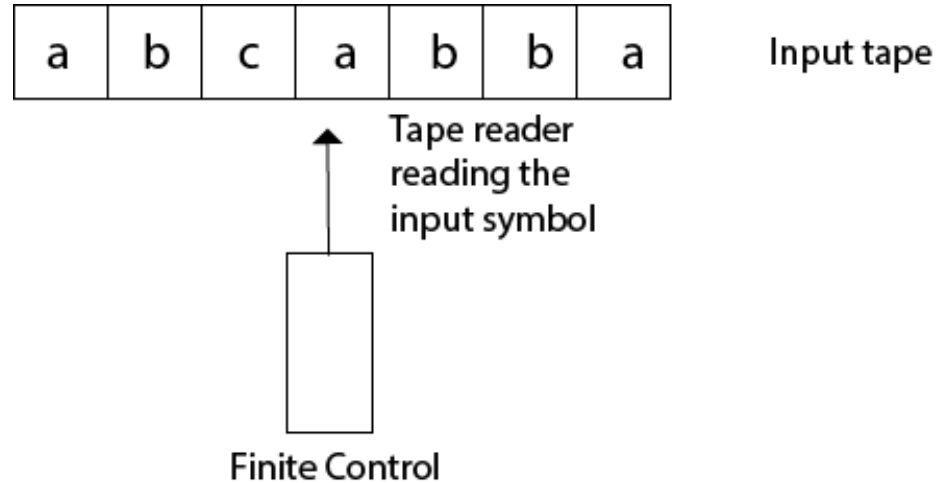
1. Deterministic Finite Automata (DFA)
2. Non-Deterministic Finite Automata (NFA)

Difference between DFA and NFA

Deterministic Finite Automata	Non Deterministic Finite Automata
From each state, for each symbol, there is exactly one transition	From each state, for each input symbol, we can have 0 or more transitions
No ϵ -transitions	ϵ -transitions are allowed

Picturization of Finite Automata

- A finite automata can be picturized as a finite control which is in some state from Q , reading a sequence of symbols from Σ written on a tape as shown below



- In one move, the finite automata does the following:
 1. Reads an input symbol
 2. Moves its head one position to the right
 3. Moves to a state as defined by δ

Deterministic Finite Automata

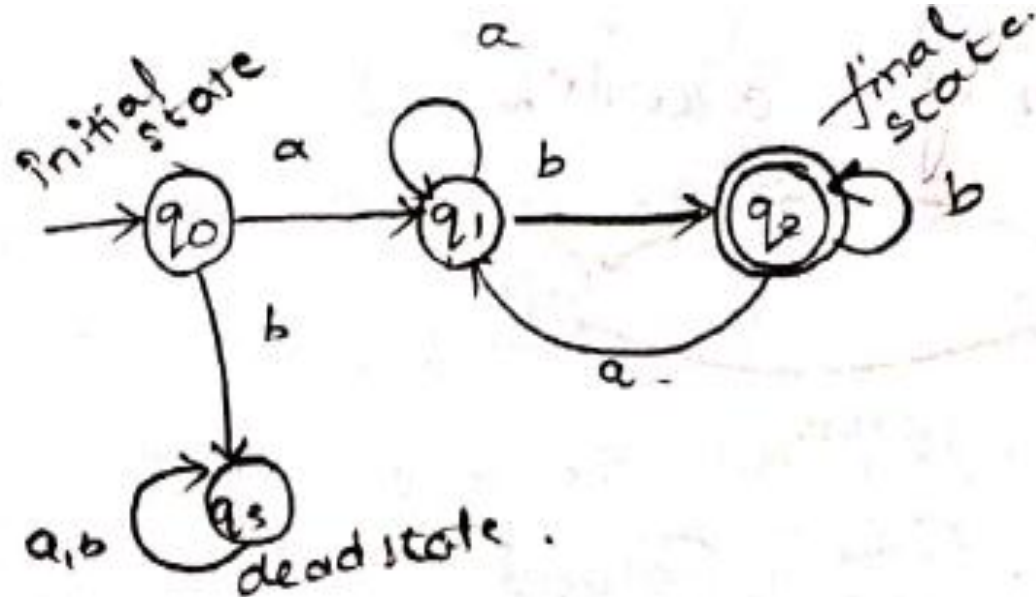
The finite automata model that allows exactly one transition from a state on an input symbol is called Deterministic Finite Automata (DFA)

An input string is accepted by a DFA if it reaches one of the final states on reading the input string

Example:

Design a DFA to accept strings that start with a and end with b over $\Sigma = \{ a, b \}$

Transition Diagram



Transition Table

	a	b
→ q0	q1	q3
q1	q1	q2
q2	q1	q2
q3	q3	q3

Formal Definition of a Finite Automata

A finite automata can be formally denoted by a five-tuple ($Q, \Sigma, \delta, q_0, F$)

where

$Q \rightarrow$ set of states

$\Sigma \rightarrow$ input alphabet

$\delta \rightarrow$ transition function : $Q \times \Sigma \rightarrow Q$

$q_0 \rightarrow$ initial state

$F \rightarrow$ set of final states, $F \subseteq Q$

EXAMPLE

	a	b
 q0	q1	q3
q1	q1	q2
 q2	q1	q2
q3	q3	q3

Formal definition of this automata is

({ q_0, q_1, q_2, q_3 }, { a, b }, $\delta, q_0, \{q_2\}$)

where δ is defined as

$\delta (q_0 , a) = q_1$

$\delta (q_0 , b) = q_3$

$\delta (q_1 , a) = q_1$

$\delta (q_1 , b) = q_2$

$\delta (q_2 , a) = q_1$

$\delta (q_2 , b) = q_2$

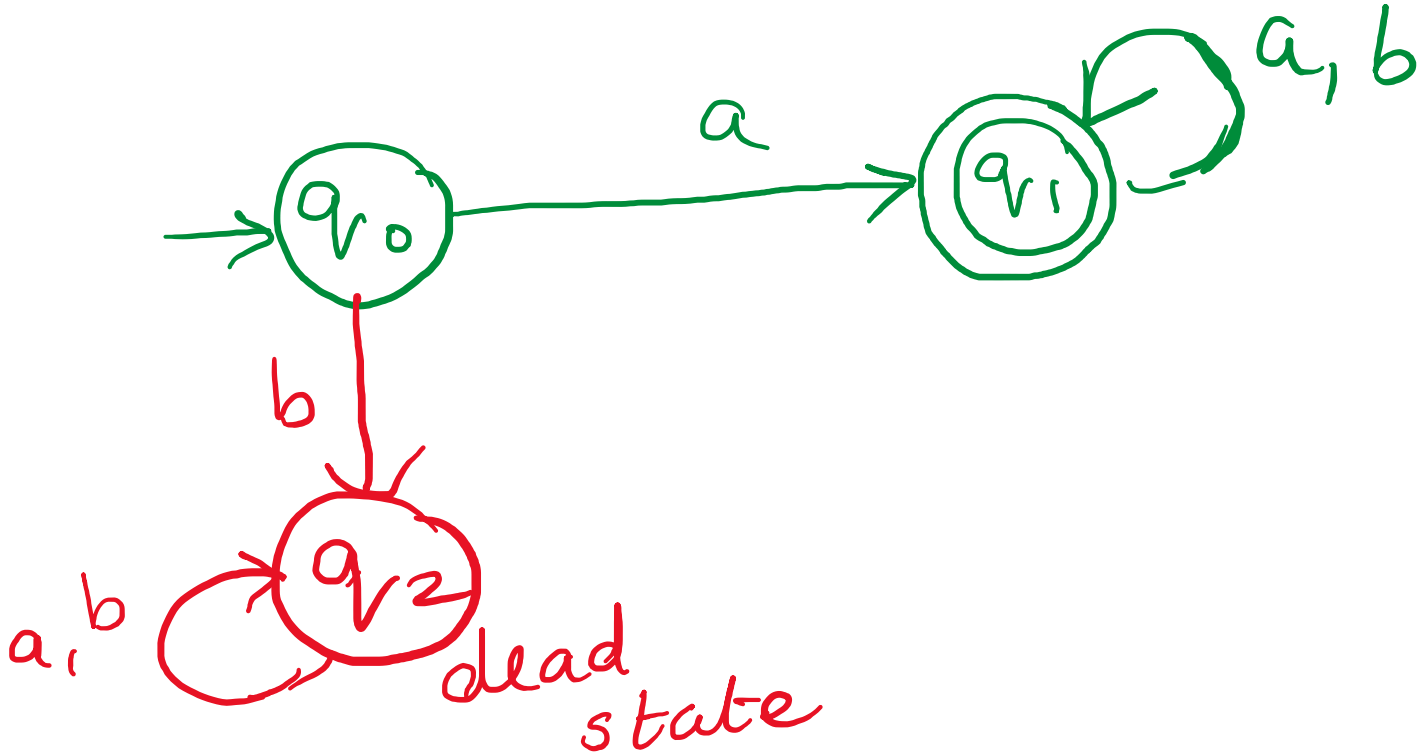
$\delta (q_3 , a) = q_3$

$\delta (q_3 , b) = q_3$

Examples : DFA

1. Design a DFA that accepts strings that start with a over $\Sigma = \{ a, b \}$

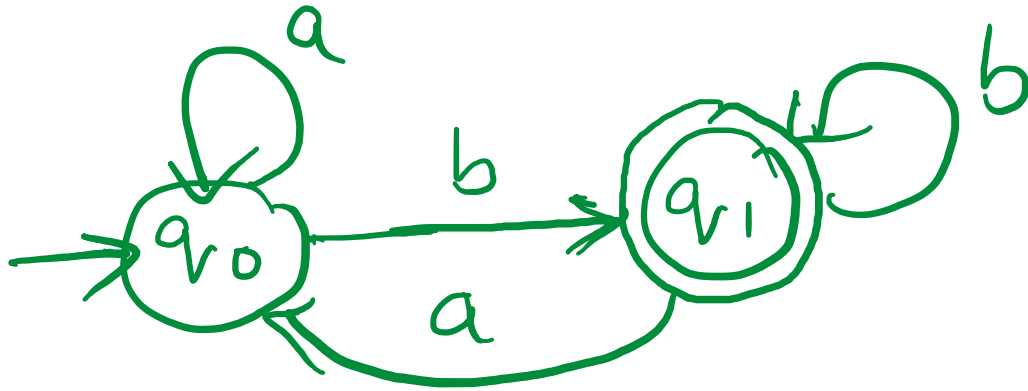
Shortest string: a



Examples : DFA

2. Design a DFA that accepts strings that end with b over $\Sigma = \{ a, b \}$

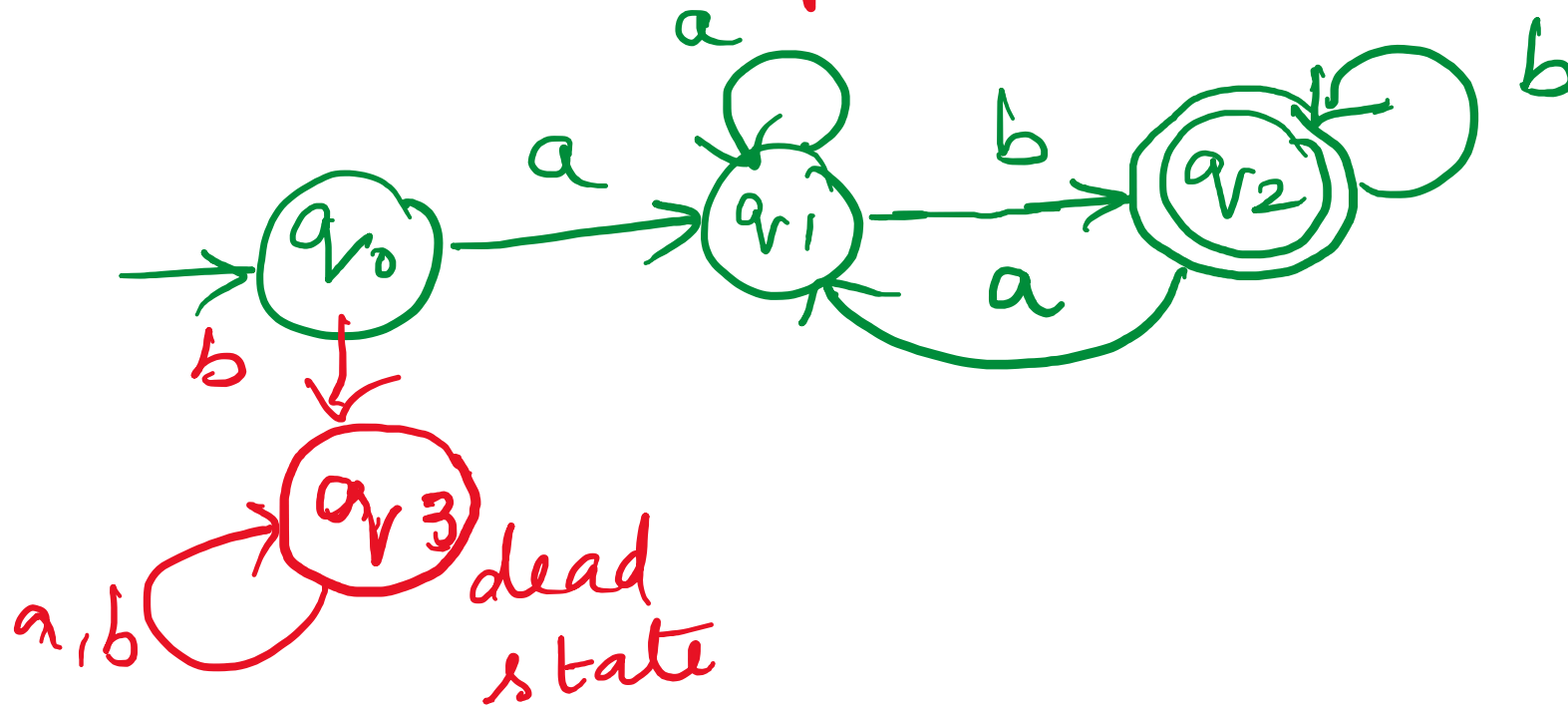
Shortest string : b



Examples : DFA

3. Design a DFA that accepts strings that start with a and end with b over $\Sigma = \{ a, b \}$

Shortest string : ab

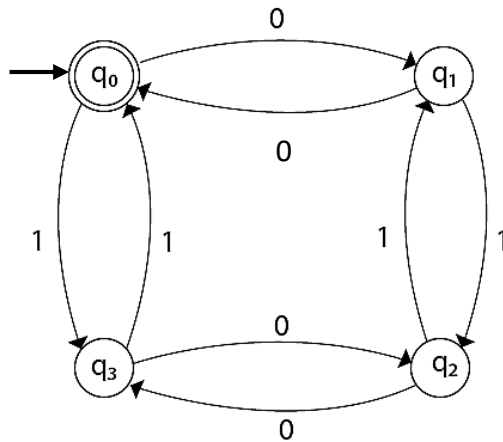


Acceptance of input strings by DFA

- A string is accepted by a DFA if and only if the DFA starting at the initial state ends in an accepting state after reading the string
- The language accepted by a DFA is the set of all strings accepted by the DFA

Example:

Check whether the string 101101 is accepted by the DFA given below. Also find the language accepted by the DFA.



Acceptance of string

$$\begin{aligned}\delta(q_0, 101101) &= \delta(q_3, 01101) \\ &= \delta(q_2, 1101) \\ &= \delta(q_1, 101) \\ &= \delta(q_2, 01) \\ &= \delta(q_3, 1) \\ &= q_0\end{aligned}$$

On reading the input 101101, the DFA ends in the final state q_0 . Hence the input is accepted.

Language Accepted by the DFA

Set of all binary strings with even number of 0s and 1s

Agenda of this Session

- Non-Deterministic Finite Automata

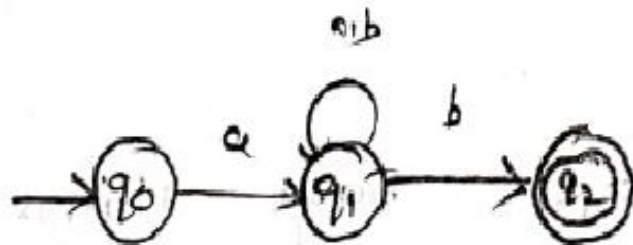
Non-Deterministic Finite Automata

- The finite automata model that allows 0 or 1 or more transitions from a state on an input symbol is called Non-Deterministic Finite Automata (NFA)
- As NFA allows more than one transition from a state, at any time, the finite control can be in any number of states.
- When a choice of next state is made, we can imagine that duplicate copies of the automata are made
- An input string will be accepted by a NFA if there is at least one path that ends in a final state

Example:

Design a NFA to accept strings that start with a and end with b over $\Sigma = \{ a, b \}$

Transition Diagram



Transition Table

	a	b
q0	q1	Φ
q1	q1	{q1,q2}
q2	Φ	Φ

Formal Definition of Finite Automata

A finite automata can be formally denoted by a five-tuple ($Q, \Sigma, \delta, q_0, F$)

where

$Q \rightarrow$ set of states

$\Sigma \rightarrow$ input alphabet

$\delta \rightarrow$ transition function $Q \times \Sigma \rightarrow P(Q)$

where $P(Q)$ is the power set of Q

$q_0 \rightarrow$ initial state

$F \rightarrow$ set of final states, $F \subseteq Q$

EXAMPLE

	a	b
q0	q1	Φ
q1	q1	{q1,q2}
q2	Φ	Φ

Formal definition of this automata is

({ q_0, q_1, q_2 }, { a, b }, $\delta, q_0, \{q_2\}$)

where δ is defined as

$\delta (q_0 , a) = q_1$

$\delta (q_0 , b) = \Phi$

$\delta (q_1 , a) = q_1$

$\delta (q_1 , b) = \{ q_1, q_2 \}$

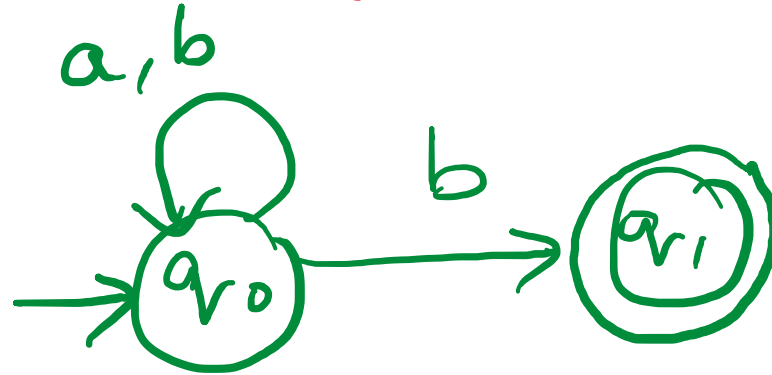
$\delta (q_2 , a) = \Phi$

$\delta (q_2 , b) = \Phi$

Examples : NFA

1. Design a NFA that accepts strings that end with b over $\Sigma = \{ a, b \}$

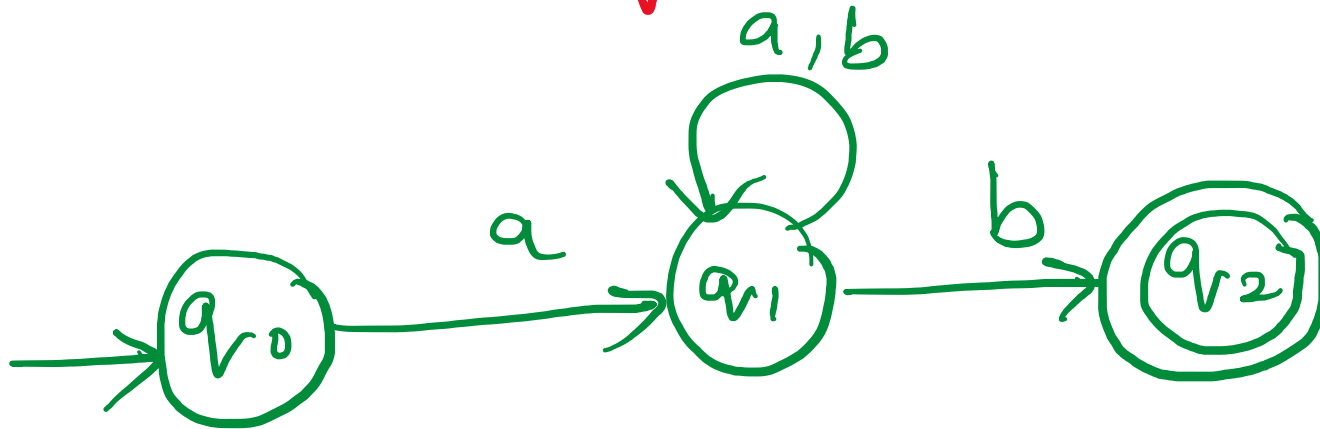
Shortest string : b



Examples : NFA

2. Design a NFA that accepts strings that start with a and end with b over $\Sigma = \{ a, b \}$

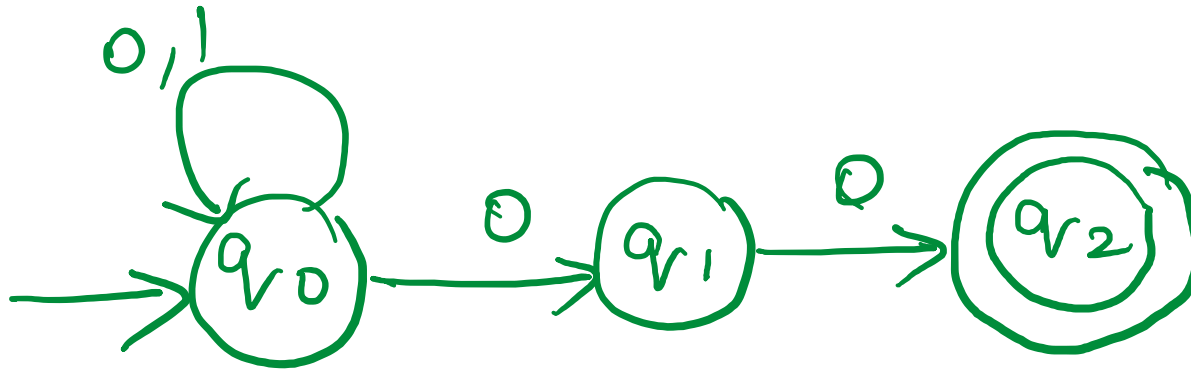
Shortest string : ab



Examples : NFA

3. Design a NFA that accepts strings that end with 00 over $\Sigma = \{ 0, 1 \}$

Shortest string : 00

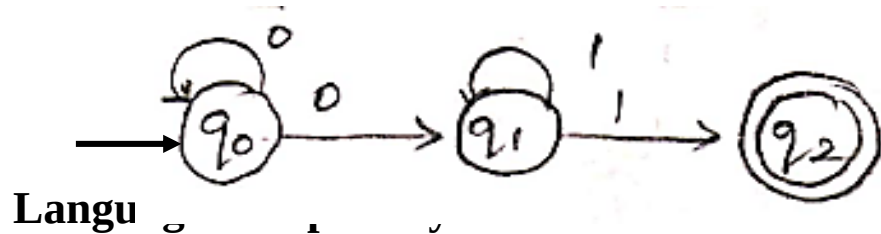


Acceptance of input strings by NFA

- An input string will be accepted by a NFA if there is at least one path that ends in a final state
- The language accepted by a NFA is the set of all strings accepted by the NFA

Example:

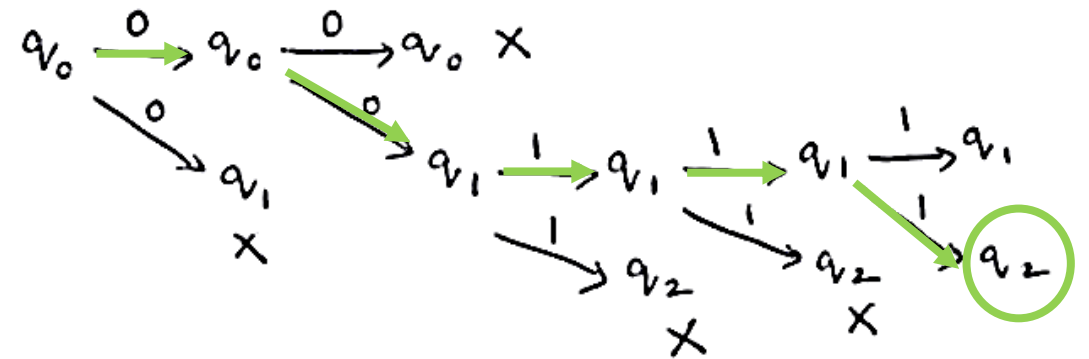
Check whether the string 00111 is accepted by the NFA given below. Also find the language accepted by the NFA.



Set of all binary strings with any number of 0s followed by any number of 1s

Acceptance of string

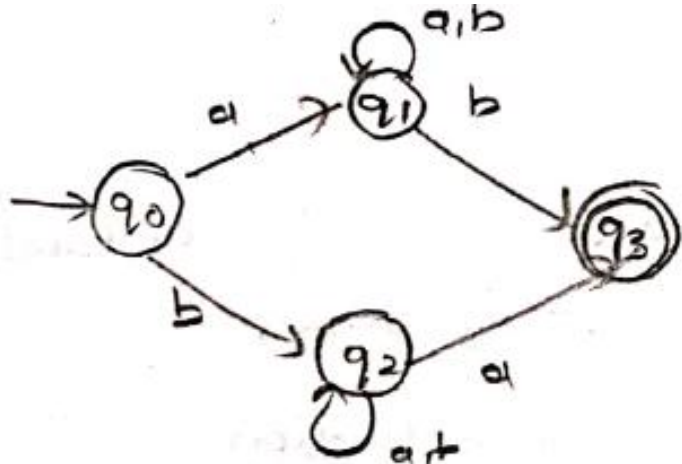
$$\begin{aligned}\delta(q_0, 00111) &= \delta(\{q_0, q_1\}, 0111) \\ &= \delta(\{q_0, q_1\}, 111) \\ &= \delta(\{q_1, q_2\}, 11) \\ &= \delta(\{q_1, q_2\}, 1) \\ &= \{q_1, q_2\}\end{aligned}$$



Simulation of NFA - Acceptance of input strings by NFA

Example:

Find the language accepted by the given NFA and simulate the given NFA for the input ababb



Language Accepted by the NFA

Set of all strings over $\Sigma = \{ a, b \}$ that start and end with different symbols

Acceptance of string

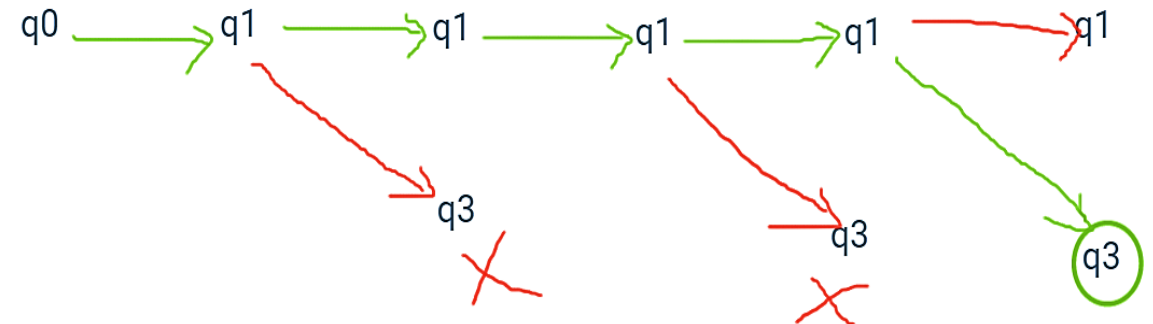
$$\delta(q_0, ababb) = \delta(\{q_1\}, babb)$$

$$= \delta(\{q_1, q_3\}, abb)$$

$$= \delta(\{q_1\}, bb)$$

$$= \delta(\{q_1, q_3\}, b)$$

$$= \{q_1, q_3\}$$



Agenda of this Session

- Finite Automaton with ϵ - moves

NFA with ϵ -moves

- ϵ is a character used to indicate null string
- A transition on ϵ means we can move from one state to another without reading any input

Example:

Transition Diagram



Transition Table

	ϵ	0	1	2
q_0	q_1	q_0	Φ	Φ
q_1	q_2	Φ	q_1	Φ
q_2	Φ	Φ	Φ	q_2

Language Accepted

Set of all strings over $\Sigma = \{ 0, 1, 2 \}$ having any number of 0's followed by any number of 1's followed by any number of 2's

Formal Definition of Finite Automata

A finite automata can be formally denoted by a

five-tuple ($Q, \Sigma, \delta, q_0, F$)

where

$Q \rightarrow$ set of states

$\Sigma \rightarrow$ input alphabet

$\delta \rightarrow$ transition function $Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$

where $P(Q)$ is the power set of Q

$q_0 \rightarrow$ initial state

$F \rightarrow$ set of final states, $F \subseteq Q$

EXAMPLE

	ϵ	0	1	2
 q0	q1	q0	Φ	Φ
q1	q2	Φ	q1	Φ
 q2	Φ	Φ	Φ	q2

Formal definition of this automata is

({ q_0, q_1, q_2 }, { 0, 1 }, $\delta, q_0, \{q_2\}$)

where δ is defined as

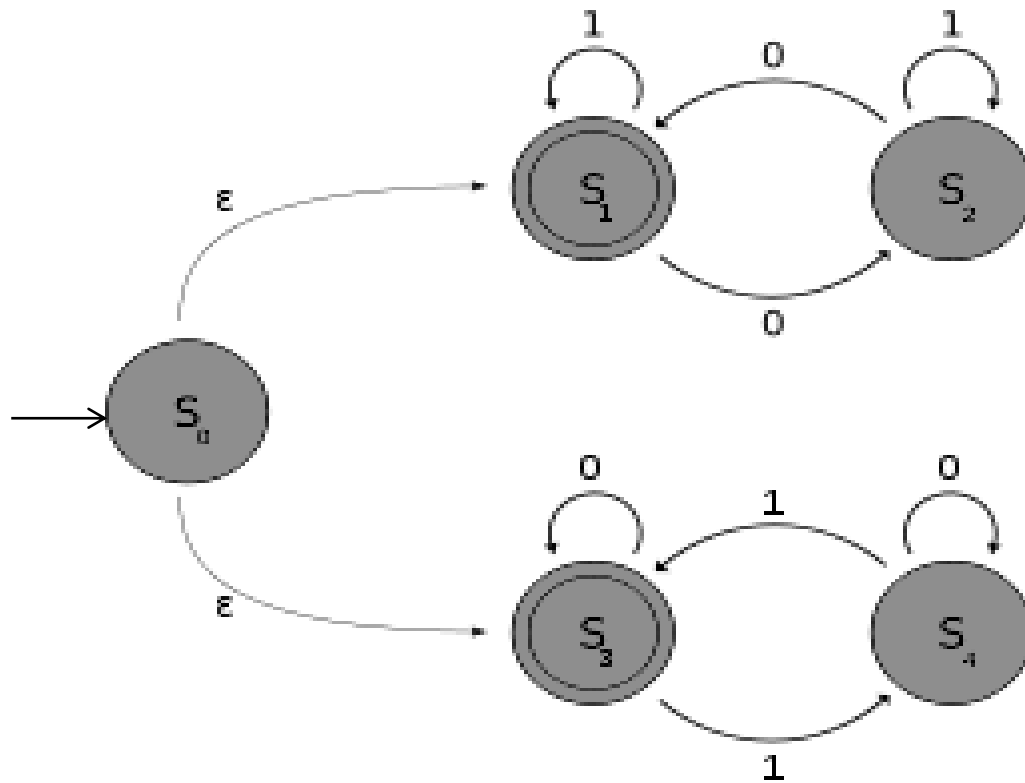
$\delta(q_0, \epsilon) = q_1$ $\delta(q_0, 0) = q_0$ $\delta(q_0, 1) = \Phi$ $\delta(q_0, 2) = \Phi$

$\delta(q_1, \epsilon) = q_2$ $\delta(q_1, 0) = \Phi$ $\delta(q_1, 1) = q_1$ $\delta(q_1, 2) = \Phi$

$\delta(q_2, \epsilon) = \Phi$ $\delta(q_2, 0) = \Phi$ $\delta(q_2, 1) = \Phi$ $\delta(q_2, 2) = q_2$

Problems related to NFA with ϵ -moves

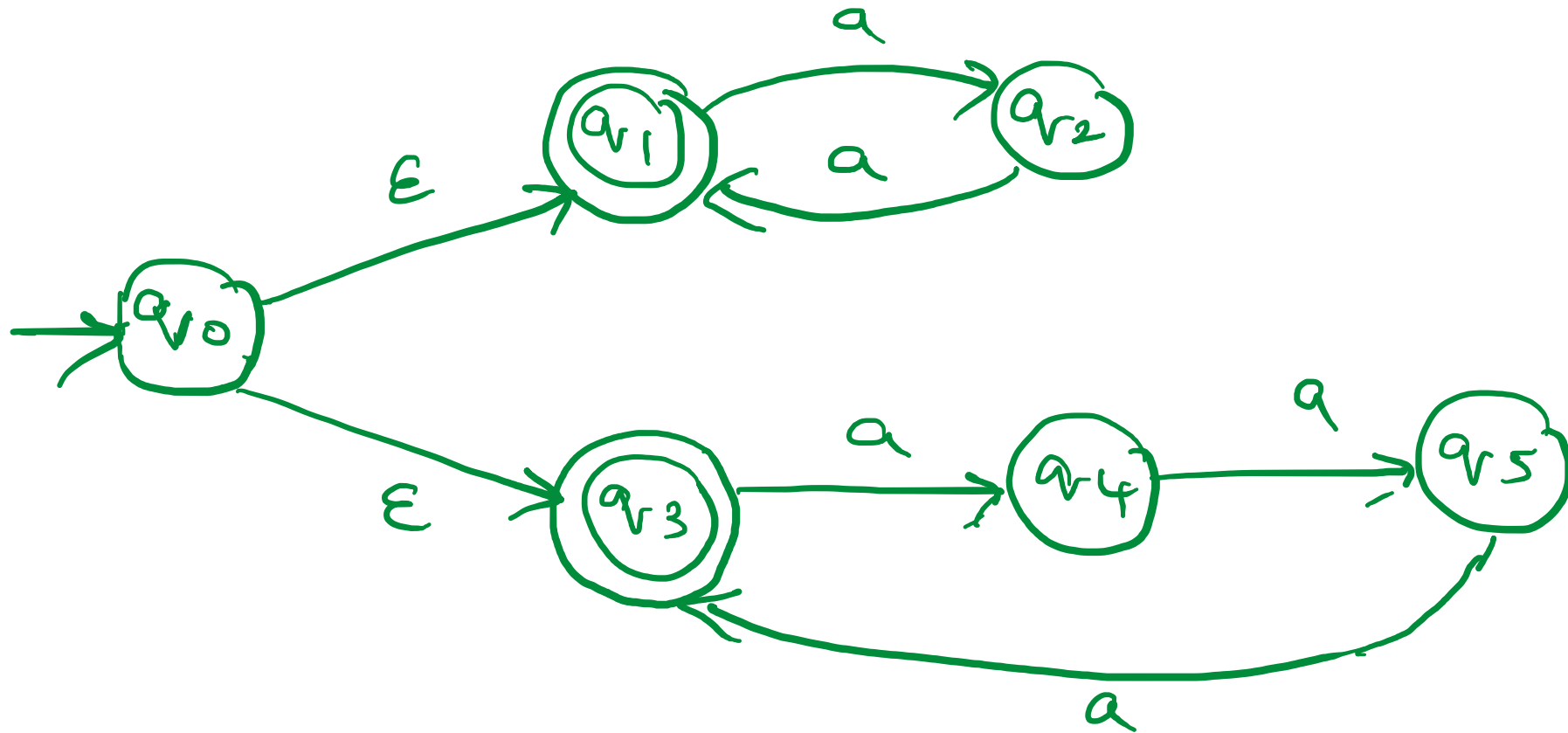
1. Design a NFA with ϵ to accept binary strings having even number of 0's or even number of 1's



Problems related to NFA with ϵ -moves

2. Design a NFA with ϵ to accept strings over $\Sigma=\{a\}$ such that

$$w = \{ a^n \mid n \text{ is even or divisible by } 3 \}$$



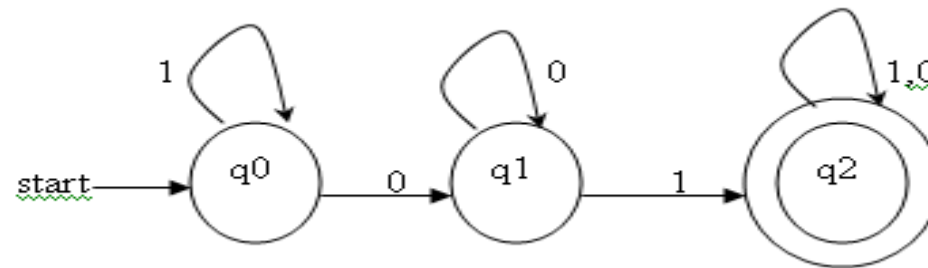
Agenda of this Session

- Problems related to Finite Automaton with and without ϵ - moves

Problems related to DFA and NFA

For the DFA given below

- a) Find the language accepted by the DFA
- b) Write the formal definition
- c) Draw the transition table
- d) Check whether the strings 10010 is accepted by the DFA



a) Set of all binary strings having 01 as a substring

b) Formal definition:

$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$
where δ is defined as

$$\delta(q_0, 0) = q_1 \qquad \delta(q_0, 1) = q_0$$

$$\delta(q_1, 0) = q_1 \qquad \delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_2 \qquad \delta(q_2, 1) = q_2$$

c. Transition Table

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

d. checking acceptance of string 10010

$$\delta(q_0, 10010) = \delta(q_0, 0010)$$

$$= \delta(q_1, 010)$$

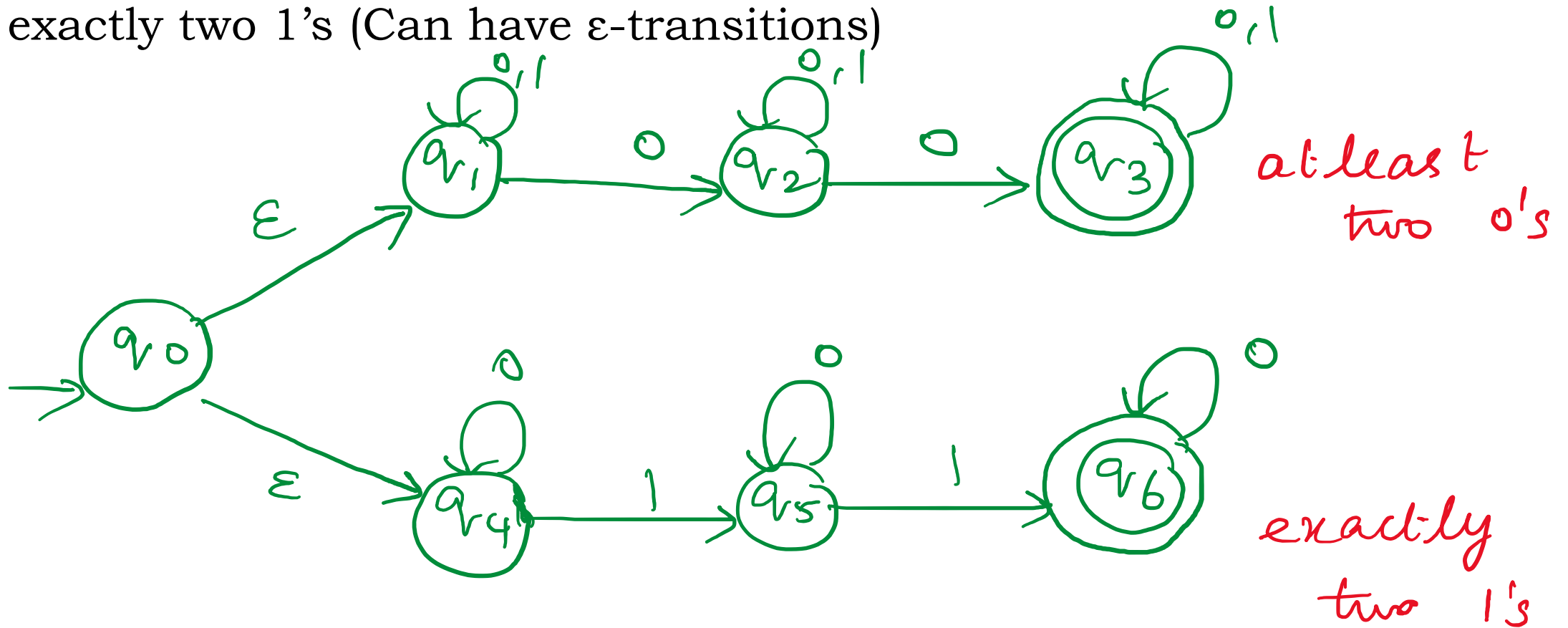
$$= \delta(q_1, 10)$$

$$= \delta(q_2, 0) = q_2 \text{ (Accepting state)}$$

I/p is accepted.

Problems related to NFA with ϵ -moves

9. Design a NFA to accept all binary strings with at least two 0's or exactly two 1's (Can have ϵ -transitions)



Agenda of this Session

- Equivalence of NFA with epsilon and DFA

NFA with ϵ -moves to DFA

THEOREM: If a language L is accepted by a nondeterministic finite automaton with ϵ -transition, then there exists a deterministic finite automaton that accepts L .

Steps:

Step 1 : Take ϵ closure for the beginning state of NFA as beginning state of DFA.

Step 2 : Find the states that can be traversed from the present for each input symbol
(union of transition value and their closures for each states of NFA present in current state of DFA).

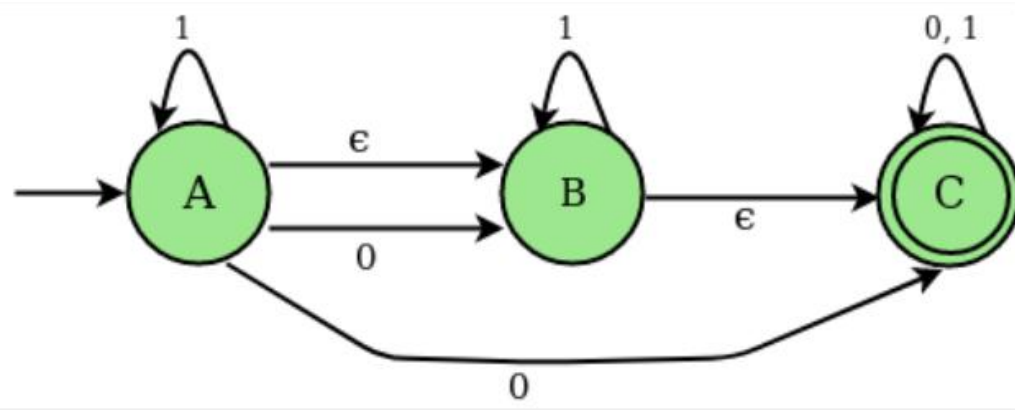
Step 3 : If any new state is found take it as current state and repeat step 2.

Step 4 : Do repeat Step 2 and Step 3 until no new state present in DFA transition table.

Step 5 : Mark the states of DFA which contains final state of NFA as final states of DFA.

Example 1

Convert the ϵ -NFA given below to a DFA



Transition table of the given NFA

	ϵ	0	1
A	B	{B,C}	A
B	C	-	B
C	-	C	C

Step 1:

Find ϵ -closure for all the states

ϵ -closure (A) = { A, B, C }

ϵ -closure (B) = { B, C }

ϵ -closure (C) = { C }

Step 2:

ϵ -closure of initial state of the NFA is the new initial state of the DFA

ϵ -closure (A) = { A, B, C } $\rightarrow$ q_0 (initial state)

From the new initial state q_0 . find the transitions for the inputs 0 and 1. If we get new states, from the new states find transitions....

Example 1 – cont...

Transition table of the given NFA

	ϵ	0	1
 A	B	{B,C}	A
B	C	-	B
C	-	C	C

$$\epsilon\text{-closure}(A) = \{ A, B, C \}$$

$$\epsilon\text{-closure}(B) = \{ B, C \}$$

$$\epsilon\text{-closure}(C) = \{ C \}$$

Let δ' be the transition function of the DFA

$$\delta'(q_0, 0) = \epsilon\text{-closure}(\delta(q_0, 0))$$

$$= \epsilon\text{-closure}(\delta(\{A, B, C\}, 0))$$

$$= \epsilon\text{-closure}(B, C)$$

$$= \epsilon\text{-closure}(B) \cup \epsilon\text{-closure}(C)$$

$$= \{B, C\} \text{ ----- } q_1$$

$$\delta'(q_0, 1) = \epsilon\text{-closure}(\delta(q_0, 1))$$

$$= \epsilon\text{-closure}(\delta(\{A, B, C\}, 1))$$

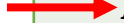
$$= \epsilon\text{-closure}(A, B, C)$$

$$= \epsilon\text{-closure}(A) \cup \epsilon\text{-closure}(B) \cup \epsilon\text{-closure}(C)$$

$$= \{A, B, C\} \text{ ----- } q_0$$

Example 1 – cont...

Transition table of the given NFA

	ϵ	0	1
 A	B	{B,C}	A
B	C	-	B
C	-	C	C

ϵ -closure (A) = { A, B, C }

ϵ -closure (B) = { B, C }

ϵ -closure (C) = { C }

$$\delta' (q_1, 0) = \epsilon\text{-closure} (\delta (q_1, 0))$$

$$= \epsilon\text{-closure} (\delta (\{B,C\} , 0))$$

$$= \epsilon\text{-closure} (C)$$

$$= \{C\} \text{ ----- } q_2$$

$$\delta' (q_1, 1) = \epsilon\text{-closure} (\delta (q_1, 1))$$

$$= \epsilon\text{-closure} (\delta (\{B,C\} , 1))$$

$$= \epsilon\text{-closure} (B,C)$$

$$= \epsilon\text{-closure}(B) \cup \epsilon\text{-closure}(C)$$

$$= \{B,C\} \text{ ----- } q_1$$

Example 1 – cont...

Transition table of the given NFA

	ϵ	0	1
 A	B	{B,C}	A
B	C	-	B
C	-	C	C

ϵ -closure (A) = { A, B, C }

ϵ -closure (B) = { B, C }

ϵ -closure (C) = { C }

$$\delta' (q_2, 0) = \epsilon\text{-closure} (\delta (q_2, 0))$$

$$= \epsilon\text{-closure} (\delta (\{C\} , 0))$$

$$= \epsilon\text{-closure} (C)$$

$$= \{C\} \text{ ----- } q_2$$

$$\delta' (q_2, 1) = \epsilon\text{-closure} (\delta (q_2, 1))$$

$$= \epsilon\text{-closure} (\delta (\{C\} , 1))$$

$$= \epsilon\text{-closure} (C)$$

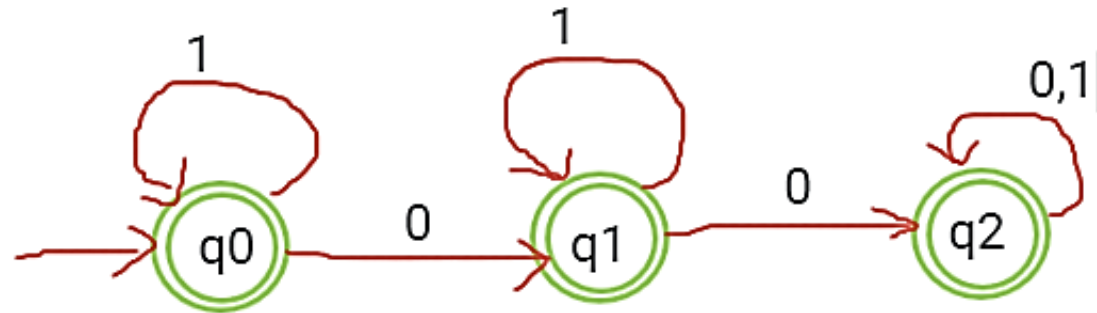
$$= \{C\} \text{ ----- } q_2$$

Example 1 – cont...

Transition table of the DFA

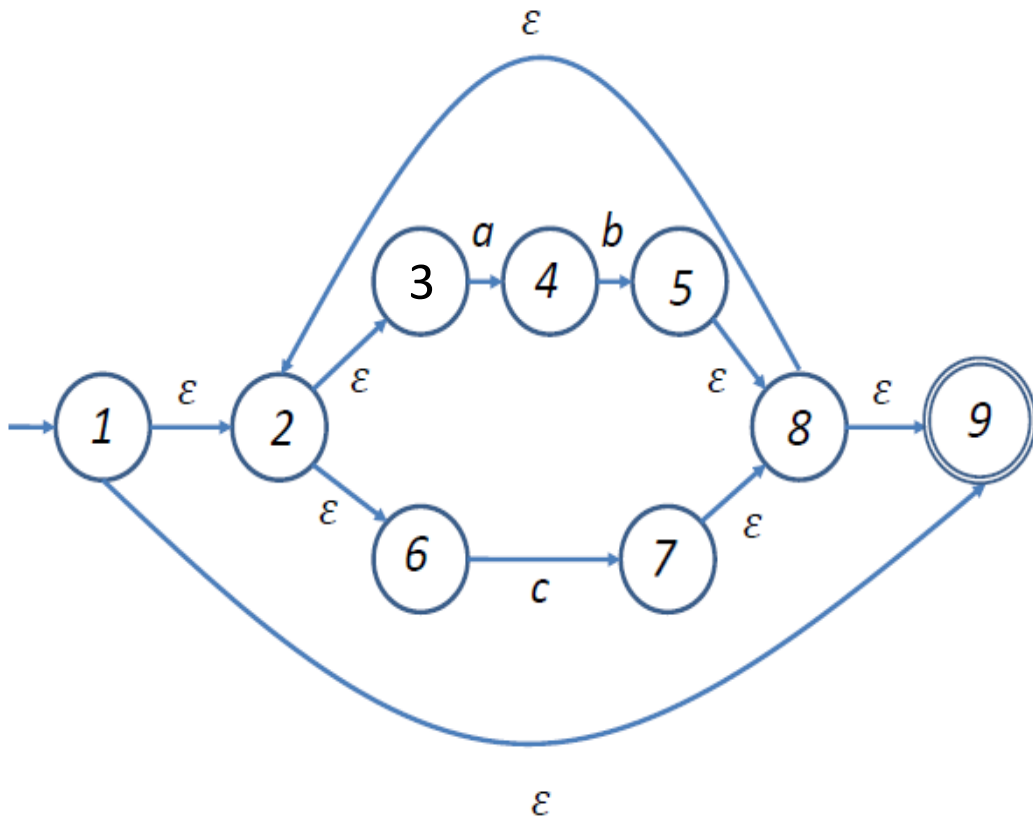
	0	1
q_0	q_1	q_0
q_1	q_2	q_1
q_2	q_2	q_2

Transition Diagram



Example 2

Convert the ϵ -NFA given below to a DFA

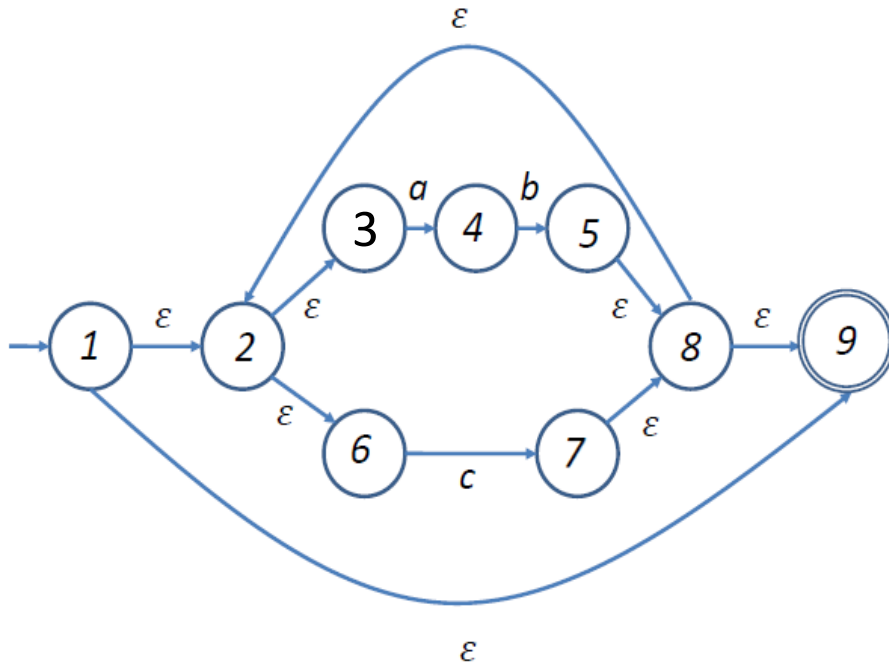


Transition table of the given NFA

	ϵ	a	b	c
→ 1	{2,9}	-	-	-
2	{3,6}	-	-	-
3	-	4	-	-
4	-	-	5	-
5	8	-	-	-
6	-	-	-	7
7	8	-	-	-
8	{2,9}	-	-	-
9	-	-	-	-

Example 2 – cont..

The given NFA



Step 1:

Find ϵ -closure for all the states

$$\epsilon\text{-closure}(1) = \{ 1, 2, 3, 6 \}$$

$$\epsilon\text{-closure}(2) = \{ 2, 3, 6 \}$$

$$\epsilon\text{-closure}(3) = \{ 3 \}$$

$$\epsilon\text{-closure}(4) = \{ 4 \}$$

$$\epsilon\text{-closure}(5) = \{ 5, 8, 2, 3, 6, 9 \}$$

$$\epsilon\text{-closure}(6) = \{ 6 \}$$

$$\epsilon\text{-closure}(7) = \{ 7, 8, 2, 3, 6, 9 \}$$

$$\epsilon\text{-closure}(8) = \{ 8, 2, 3, 6, 9 \}$$

$$\epsilon\text{-closure}(9) = \{ 9 \}$$

Example 2 – cont..

Transition table of the given NFA

	ϵ	a	b	c
→ 1	{2,9}	-	-	-
2	{3,6}	-	-	-
3	-	4	-	-
4	-	-	5	-
5	8	-	-	-
6	-	-	-	7
7	8	-	-	-
8	{2,9}	-	-	-
9	-	-	-	-

Let δ' be the transition function of the DFA

$$\delta' (A, a) = \epsilon\text{-closure} (\delta (A, a))$$

$$= \epsilon\text{-closure} (\delta (\{1,2,3,6\} , a))$$

$$= \epsilon\text{-closure} (4)$$

$$= \{ 4 \} \text{ ----- B}$$

$$\delta' (A, b) = \epsilon\text{-closure} (\delta (A, b))$$

$$= \epsilon\text{-closure} (\delta (\{1,2,3,6\} , b))$$

$$= \epsilon\text{-closure} (\Phi)$$

$$= \Phi$$

$$\delta' (A, c) = \epsilon\text{-closure} (\delta (A, c))$$

$$= \epsilon\text{-closure} (\delta (\{1,2,3,6\} , c))$$

$$= \epsilon\text{-closure} (7)$$

$$= \{ 7, 8, 2, 3, 6, 9 \} \text{ ----- C}$$

Example 2 – cont..

Transition table of the given NFA

	ϵ	a	b	c
 1	{2,9}	-	-	-
2	{3,6}	-	-	-
3	-	4	-	-
4	-	-	5	-
5	8	-	-	-
6	-	-	-	7
7	8	-	-	-
8	{2,9}	-	-	-
9	-	-	-	-

$$\delta' (B, a) = \epsilon\text{-closure} (\delta (B, a))$$

$$= \epsilon\text{-closure} (\delta (\{4\} , a))$$

$$= \epsilon\text{-closure} (\Phi)$$

$$= \Phi$$

$$\delta' (B, b) = \epsilon\text{-closure} (\delta (B, b))$$

$$= \epsilon\text{-closure} (\delta (\{4\} , b))$$

$$= \epsilon\text{-closure} (5)$$

$$= \{ 5, 8, 2, 3, 6, 9 \} \text{ ----- D}$$

$$\delta' (B, c) = \epsilon\text{-closure} (\delta (B, c))$$

$$= \epsilon\text{-closure} (\delta (\{4\} , c))$$

$$= \epsilon\text{-closure} (\Phi)$$

$$= \Phi$$

Example 2 – cont..

Transition table of the given NFA

	ϵ	a	b	c
 1	{2,9}	-	-	-
2	{3,6}	-	-	-
3	-	4	-	-
4	-	-	5	-
5	8	-	-	-
6	-	-	-	7
7	8	-	-	-
8	{2,9}	-	-	-
 9	-	-	-	-

$$\delta' (C, a) = \epsilon\text{-closure} (\delta (C, a))$$

$$= \epsilon\text{-closure} (\delta (\{7,8,2,3,6,9\} , a))$$

$$= \epsilon\text{-closure} (4)$$

$$= \{ 4 \} \text{ ----- B}$$

$$\delta' (C, b) = \epsilon\text{-closure} (\delta (C, b))$$

$$= \epsilon\text{-closure} (\delta (\{7,8,2,3,6,9\} , b))$$

$$= \epsilon\text{-closure} (\Phi)$$

$$= \Phi$$

$$\delta' (C, c) = \epsilon\text{-closure} (\delta (C, c))$$

$$= \epsilon\text{-closure} (\delta (\{7,8,2,3,6,9\} , c))$$

$$= \epsilon\text{-closure} (7)$$

$$= \{ 7, 8, 2, 3, 6, 9 \} \text{ ----- C}$$

Example 2 – cont..

Transition table of the given NFA

	ϵ	a	b	c
 1	{2,9}	-	-	-
2	{3,6}	-	-	-
3	-	4	-	-
4	-	-	5	-
5	8	-	-	-
6	-	-	-	7
7	8	-	-	-
8	{2,9}	-	-	-
 9	-	-	-	-

$$\delta' (D, a) = \epsilon\text{-closure} (\delta (D, a))$$

$$= \epsilon\text{-closure} (\delta (\{5,8,2,3,6,9\} , a))$$

$$= \epsilon\text{-closure} (4)$$

$$= \{ 4 \} \text{ ----- B}$$

$$\delta' (D, b) = \epsilon\text{-closure} (\delta (D, b))$$

$$= \epsilon\text{-closure} (\delta (\{5,8,2,3,6,9\} , b))$$

$$= \epsilon\text{-closure} (\Phi)$$

$$= \Phi$$

$$\delta' (D, c) = \epsilon\text{-closure} (\delta (D, c))$$

$$= \epsilon\text{-closure} (\delta (\{5,8,2,3,6,9\} , c))$$

$$= \epsilon\text{-closure} (7)$$

$$= \{ 7, 8, 2, 3, 6, 9 \} \text{ ----- C}$$

Example 2 – cont...

Transition table of the DFA

	a	b	c
→ A	B	-	C
B	-	D	-
C	B	-	C
D	B	-	C

Transition Diagram

